

Il dato astratto *Vettore*

Un esempio elementare di specifica assiomatica è quella di un vettore

a) Specifica sintattica

Tipi: vettore, intero, tipoelem

Operatori:

CREAVETTORE() → vettore

LEGGIVETTORE(vettore, intero) → tipoelem

SCRIVIVETTORE(vettore, intero, tipoelem) → vettore

Il dato astratto *Vettore*

b) Specifica semantica

Tipi:

intero: l'insieme dei numeri interi

vettore: l'insieme delle sequenze di n elementi di tipo *tipoelem*

Operatori:

CREAVETTORE = v

Pre: non ci sono precondizioni, o più precisamente, il predicato che definisce la precondizione di applicabilità di CREAVETTORE è il valore VERO

*Post: $\forall i \in \{0, 1, 2, \dots, n-1\}$, l' i -esimo elemento del vettore, $v(i)$, è uguale ad un prefissato elemento di tipo *tipoelem**

LEGGIVETTORE(v, i) = e

Pre: $0 \leq i \leq n-1$

Post: $e = v(i)$

SCRIVIVETTORE(v, i, e) = v'

Pre: $0 \leq i \leq n-1$

Post: $\forall j \in \{0, 1, \dots, n-1\}, j \neq i, v'(j) = v(j), v'(i) = e$

Il dato astratto *Vettore*

c) Realizzazione

In Java il vettore (array) è un tipo di dato concreto. La corrispondenza tra la specifica appena introdotta e quella del Java è la seguente:

CREAVETTORE $\Leftrightarrow$ tipoelem $v[n]$

LEGGIVETTORE(v, i) $\Leftrightarrow v[i]$

SCRIVIVETTORE(v, i, e) $\Leftrightarrow v[i] = e$

dove i può essere anche un'espressione di tipo intero

Il dato astratto *Pila*

Una pila (*stack*) è una sequenza di elementi di un certo tipo in cui è possibile aggiungere o togliere elementi solo da un estremo della sequenza (la “testa”).

Può essere vista come un caso speciale di lista in cui l'ultimo elemento inserito è il primo ad essere rimosso (LIFO, *last in, first out*) e non è possibile accedere ad alcun elemento che non sia quello in testa.

Il dato astratto *Pila*

Una specifica assiomatica di una pila

Specifica sintattica

Tipi: pila, booleano, tipoelem

Operatori:

CREAPILA() $\rightarrow$ pila

PILAVUOTA(pila) $\rightarrow$ booleano

LEGGIPILA(pila) $\rightarrow$ tipoelem

FUORIPILA(pila) $\rightarrow$ pila

INPILA(tipoelem, pila) $\rightarrow$ pila

Il tipo astratto *Pila*

Specifica semantica

Tipi:

pila = insieme delle sequenze di elementi di tipo
tipoelem

booleano = insieme dei valori di verità {vero,falso}

Operatori:

CREAPILA ()=P'

Post: $P' = \wedge$, la sequenza vuota

PILAVUOTA(P)=b

Post: $b = \text{vero}$, se $P = \wedge$; $b = \text{falso}$, altrimenti

Il tipo astratto *Pila*

LEGGIPILA(P)=a

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $a = a_1$

FUORIPILA(P)=P'

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $P' = a_2, \dots, a_n$ se $n > 1$; $P' = \wedge$ se $n = 1$

INPILA(a,P)=P'

Pre: $P = a_1, a_2, \dots, a_n$ e $n \geq 0$

Post: $P' = a, a_1, a_2, \dots, a_n$ se $n > 0$; $P' = a$ se $n = 0$

Il dato astratto *Coda*

Una coda è un dato astratto che consente di rappresentare una sequenza di elementi in cui è possibile aggiungere elementi ad un estremo (“il fondo”) e togliere elementi dall’altro estremo (“la testa”).

Tale disciplina di accesso è detta FIFO (**First In First Out**).

È adatta a rappresentare sequenze nelle quali l’elemento viene elaborato secondo l’ordine di arrivo (lista d’attesa, etc.)

Il dato astratto *Coda*

Una specifica assiomatica di una coda

Specifica sintattica

Tipi: coda, booleano, tipoelem

Operatori:

CREACODA() $\rightarrow$ coda

CODAVUOTA(coda) $\rightarrow$ booleano

LEGGICODA(coda) $\rightarrow$ tipoelem

FUORICODA(coda) $\rightarrow$ coda

INCODA(tipoelem, coda) $\rightarrow$ coda

Il dato astratto *Coda*

Specifica semantica

Tipi:

coda = insieme delle sequenze $\langle a_1, a_2, \dots, a_n \rangle$, con $n > 0$, di elementi di tipo *tipoelem*. Λ denota la sequenza vuota.

booleano = insieme dei valori di verità {vero, falso}

Operatori:

CREACODA() $=q'$

Post: $q' = \Lambda$

CODAVUOTA(q) $=b$

Post: $b = \text{vero}$, se $q = \Lambda$; $b = \text{falso}$, altrimenti

LEGGICODA(q) $=a$

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_1$

Il dato astratto *Coda*

Specifica semantica (cont.)

FUORICODA(q)= q'

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_2, \dots, a_n \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

INCODA(a, q)= q'

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_n, a \rangle$, se $q \neq \Lambda$;

$q' = \langle a \rangle$ se $q = \Lambda$

Esercizio

Si vuole progettare un dato astratto *Deque* (*double-ended queue* o *coda doppia*), una struttura lineare che permette di inserire, cancellare, esaminare elementi solo dalle due estremità.

Dare le specifiche assiomatiche (semantiche) di *deque*, tenuto conto delle seguenti specifiche sintattiche:

Esercizio

Tipi

deque, item, boolean

Operatori

<code>new()</code>	$\rightarrow$ deque	crea nuove code doppie
<code>addq(item, deque)</code>	$\rightarrow$ deque	accoda un elemento
<code>bottom(deque)</code>	$\rightarrow$ item	restituisce l'elemento di coda
<code>leaveq(deque)</code>	$\rightarrow$ deque	estrae l'elemento di coda
<code>push(item, deque)</code>	$\rightarrow$ deque	impila un elemento
<code>top(deque)</code>	$\rightarrow$ item	restituisce l'elemento di testa
<code>pop(deque)</code>	$\rightarrow$ deque	estrae l'elemento di testa
<code>isnew(deque)</code>	$\rightarrow$ boolean	stabilisce se una coda è vuota

Esercizio

Specifica semantica

Tipi:

deque = insieme delle sequenze $\langle a_1, a_2, \dots, a_n \rangle$, con $n > 0$, di elementi di tipo *item*. Λ denota la sequenza vuota.

boolean = insieme dei valori di verità {vero, falso}

Operatori:

new () = q'

Post: $q' = \Lambda$

Esercizio

Specifica semantica (cont.)

$\text{addq}(a, q) = q'$ // accoda un elemento

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_n, a \rangle$, se $q \neq \Lambda$;
 $q' = \langle a \rangle$ se $q = \Lambda$

$\text{bottom}(q) = a$ // restituisce l'elemento di coda

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_n$

$\text{leaveq}(q) = q'$ //estrae l'elemento di coda

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_1, a_2, \dots, a_{n-1} \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

Esercizio

Specifica semantica (cont.)

$\text{push}(a, q) = q'$ // impila un elemento

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle$ oppure $q = \Lambda$

Post: $q' = \langle a, a_1, a_2, \dots, a_n \rangle$, se $q \neq \Lambda$;
 $q' = \langle a \rangle$ se $q = \Lambda$

$\text{top}(q) = a$ // restituisce l'elemento di testa

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $a = a_1$

$\text{pop}(q) = q'$ // estrae l'elemento di testa

Pre: $q = \langle a_1, a_2, \dots, a_n \rangle \neq \Lambda$

Post: $q' = \langle a_2, \dots, a_n \rangle$ se $n > 1$; $q' = \Lambda$ se $n = 1$

Esercizio

Specifica semantica (cont.)

isnew(q)=b // stabilisce se una coda doppia è vuota

Post: b=vero, se $q=\wedge$; b=falso, altrimenti

Esercizio

Si vuole progettare il dato astratto *volo*.

Dare le specifiche assiomatiche (semantiche) di *volo*, tenuto conto delle seguenti specifiche sintattiche

Esercizio

Tipi

volo, passeggero, posto, integer, boolean

Operatori

creavolo() → volo crea un nuovo volo

postolibero(posto, volo) → boolean stabilisce se il posto è libero
o meno

check-in(posto, passeggero, volo) → volo aggiunge un passeggero
al volo assegnandogli il posto specificato, purché libero

libera(posto, volo) → volo libera un posto sul volo

sostituisci(posto, passeggero, volo) → volo assegna a un altro
passeggero un posto già occupato

passeggero(posto, volo) → passeggero restituisce il passeggero
che occupa un posto

passeggeri(volo) → integer restituisce il numero di passeggeri
con carta di imbarco, cioè che hanno fatto il check-in

Esercizio

Suggerimento:

posto è l'insieme K dei posti

passaggero è l'insieme P dei passeggeri.

volo è l'insieme di voli, dove ogni volo è a sua volta un insieme di coppie $(k, p) \in K \times P$.

integer è l'insieme Z dei numeri interi. 0 e 1 sono particolari valori dell'insieme Z .

boolean è l'insieme dei valori di verità $\{\text{true}, \text{false}\}$.

N.B.: Questa definizione è sufficientemente astratta e non dice nulla sull'insieme K . Infatti i posti su un aereo sono generalmente identificati da una coppia

Numero Lettera

Esercizio

Specifica semantica

Tipi:

volo = insieme di coppie $(k,p) \in K \times P$ dove K è l'insieme dei posti e P è l'insieme dei passeggeri. $\emptyset$ denota l'insieme vuoto.

integer = insieme di numeri naturali

boolean = insieme dei valori di verità {vero, falso}

Operatori:

creavolo() = v' // crea un nuovo volo

Post: $v' = \emptyset$

Esercizio

Specifica semantica (cont.)

postolibero(k, v) = b // stabilisce se il posto è libero o meno

Pre: $v \subseteq K \times P, k \in K$

Post: $b = \text{vero}$ se $\neg(\exists p \in P (k, p) \in v)$, falso altrimenti

check-in(k, p, v) = v' // aggiunge un passeggero al volo
assegnandogli il posto specificato, purché libero

Pre: $v \subseteq K \times P, k \in K, p \in P \neg(\exists q \in P (k, q) \in v)$,

Post: $v' = v \cup \{(k, p)\}$

Esercizio

Specifica semantica (cont.)

$\text{libera}(k, v) = v'$ // libera un posto sul volo

Pre: $v \subseteq K \times P, k \in K, \exists q \in P (k, q) \in v$

Post: $v' = v - \{(k, q)\}$

$\text{sostituisci}(k, p, v) = v'$ // assegna a un altro passeggero un posto già
occupato

Pre: $v \subseteq K \times P, k \in K, p \in P, \exists q \in P (k, q) \in v$

Post: $v' = (v - \{(k, q)\}) \cup \{(k, p)\}$

Esercizio

Specifica semantica (cont.)

passaggero(k, v)=p //restituisce il passeggero che occupa un posto

Pre: $v \subseteq K \times P$, $k \in K$, $\exists q \in P (k, q) \in v$

Post: $p=q$

passaggeri(v)=n //restituisce il numero di passeggeri con carta di imbarco, cioè che hanno fatto il check-in

Pre: $v \subseteq K \times P$

Post: $n=|v|$

Specifica algebrica di *Pila*

Specifica sintattica

sorts: stack, item, boolean

operations:

newstack() -> stack

push(stack, item) -> stack

pop(stack) -> stack

top(stack) -> item

isnew(stack) -> boolean

Ogni insieme è detto un **sort** (letteralmente “tipo”) dell'algebra eterogenea.

I sort **item** e **boolean** sono **ausiliari** alla definizione di stack.

Specifica algebrica di *Pila*

Specifica semantica

declare **stk**: stack, **i**: item

pop(push(stk, i)) = stk

top(push(stk, i)) = i

isnew(newstack) = true

isnew(push(stk, i)) = false

Nella specifica semantica
occorre **dichiarare** i
parametri su cui lavorano
gli operatori.

Specifica algebrica di *Pila*

Specifica di restrizione

restrictions

$\text{pop}(\text{newstack}) = \text{error}$

$\text{top}(\text{newstack}) = \text{error}$

dove 'error' è un elemento speciale indefinito.

Specifica algebrica di *Pila*

Ovviamente avremmo potuto scrivere molte altre equazioni, come:

$$isnew(pop(push(newstack, i))) = true$$

che evidenzia come il predicato *isnew* sia vero anche per quegli stack ottenuti inserendo un generico elemento *i* in un **nuovo** stack e poi rimuovendolo.

Tuttavia questa equazione è **ridondante**, poiché è ricavabile dalle altre scritte in precedenza. Infatti, grazie alla prima e terza equazione possiamo scrivere:

$$isnew(pop(push(newstack, i))) = isnew(newstack) = true$$

Costruttori e Osservazioni

<i>osservazioni</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>pop(stk')</i>	error	stk
<i>top(stk')</i>	error	i
<i>isnew(stk')</i>	true	false

Osservazione binaria: $\text{equal}(l, m)$

Esempio: predicato $\text{equal}(l, m)$ che è vero solo se le due pile contengono gli stessi elementi nello stesso ordine.

<i>Costruttore di m</i>	<i>Costruttore di l</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>newstack</i>	true	false
<i>push(stk', i')</i>	false	$i=i'$ and $\text{equal}(\text{stk}, \text{stk}')$

Questa tabella può essere vista come l'aggiunta di una terza dimensione alla tabella di base per le osservazioni unarie.

Osservazione binaria: $\text{equal}(\text{stk}', m)$

Questa terza dimensione può essere rimossa andando a classificare esplicitamente solo il primo argomento dell'osservazione e referenziando in modo astratto il secondo argomento mediante una **variabile libera**.

<i>osservazione</i>	<i>Costruttore di stk'</i>	
	<i>newstack</i>	<i>push(stk, i)</i>
<i>equal(stk', m)</i>	isnew(m)	not isnew(m) and i=top(m) and equal(stk, pop(m))

Specifica algebrica di coda

Specifica sintattica

sorts: queue, item, boolean

operations:

newq() $\rightarrow$ queue

addq(queue, item) $\rightarrow$ queue

deleteq(queue) $\rightarrow$ queue

frontq(queue) $\rightarrow$ item

isnewq(queue) $\rightarrow$ boolean

Richiamo: Specifica assiomatica di *Coda*

Specifica semantica

Operatori:

CREACODA() $=Q'$

Post: $Q' = \wedge$, la sequenza vuota

CODAVUOTA(Q) $=b$

Post: $b = \text{vero}$, se $Q = \wedge$; $b = \text{falso}$, altrimenti

LEGGICODA(Q) $=a$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $a = a_1$

FUORICODA(Q) $=Q'$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 1$

Post: $Q' = a_2, \dots, a_n$ se $n > 1$; $Q' = \wedge$ se $n = 1$

INCODA(a,Q) $=Q'$

Pre: $Q = a_1, a_2, \dots, a_n$ e $n \geq 0$

Post: $Q' = a_1, a_2, \dots, a_n, a$ se $n > 0$; $Q' = a$ se $n = 0$

Costruttori e Osservazioni

<i>osservazioni</i>	<i>Costruttore di q'</i>	
	<i>newq</i>	<i>addq(q, i)</i>
<i>isnew(q')</i>	true	false
<i>frontq(q')</i>	error	if isnewq(q) then i else frontq(q)
<i>deleteq(q')</i>	error	if isnewq(q) then newq else addq(delete(q), i)

Specifica algebrica di coda

Specifica semantica

declare **q**, **r**: queue, **i**: item

isnewq(newq) = true

isnewq(addq(q, i)) = false

deleteq(addq(q, i)) = **if** isnewq(q) **then** newq **else** addq(deleteq(q), i)

frontq(addq(q, i)) = **if** isnewq(q) **then** i **else** frontq(q)

Specifica algebrica di coda

Specifica di restrizione

restrictions

frontq(newq) = error

deleteq(newq) = error

Esercizio

- Fornire una specifica algebrica completa, consistente e minimale (cioè non ridondante) del dato astratto *Vettore* di cui è stata fornita la specifica assiomatica mediante pre- e postcondizioni.

sorts: tipoelem, array, integer

operations:

new() -> array

read(array, integer) -> tipoelem

write(array, integer, tipoelem) -> array

N.B. Supponiamo che l'array sia inizializzato con il valore 0.

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di v'</i>	
	<i>new()</i>	<i>write(v, i, e)</i>
<i>read(v', i')</i>	0	if i=i' then e else read(v, i')

Specifica semantica

declare: **i, j**: integer; **e**: tipoelem; **v**: array

read(new()) = 0

read(write(v, i, e), j) = **if** i=j **then** e **else** read(v, j)

Esercizio (cont.)

Miglioriamo la specifica di vettore con l'introduzione del controllo sugli indici.

Specifica sintattica

sorts: array, integer, tipoelem

operations:

create(integer, integer) -> array //crea un vettore con lower/upper bound

assign(array, integer, tipoelem) -> array

first(array) -> integer // restituisce lower bound

last(array) -> integer // restituisce upper bound

eval(array, integer) -> tipoelem // restituisce l'elemento

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di v'</i>	
	<i>create(x, y)</i>	<i>assign(v, i, e)</i>
<i>first(v')</i>	x	<i>first(v)</i>
<i>last(v')</i>	y	<i>last(v)</i>
<i>eval(v', i')</i>	error	<i>if $i' < \text{first}(a)$ or $i' > \text{last}(a)$ then error else if $i' = i$ then e else eval(v, i')</i>

Esercizio (cont.)

Specifica semantica

declare: **x, y, i, j**: integer; **e**: tipoelem; **v**: array

$\text{first}(\text{create}(x, y)) = x$

$\text{first}(\text{assign}(v, i, e)) = \text{first}(v)$

$\text{last}(\text{create}(x, y)) = y$

$\text{last}(\text{assign}(v, i, e)) = \text{last}(v)$

$\text{eval}(\text{create}(x, y), i) = \mathbf{error}$

$\text{eval}(\text{assign}(v, i, e), j) = \mathbf{if } j < \text{first}(v) \text{ or } j > \text{last}(v) \text{ then } \mathbf{error} \text{ else if } i = j \text{ then } e \text{ else}$

$\text{eval}(v, j)$

Esercizio

- Fornire una specifica algebrica semantica completa, consistente e minimale per il dato astratto *Stringa* supposto che le specifiche sintattiche siano le seguenti:

sorts: string, char, integer, boolean

operations:

new() → string crea nuove stringhe

append(string, string) → string concatena due stringhe

add(string, char) → string aggiunge un carattere a fine stringa

length(string) → integer calcola la lunghezza di una stringa

isEmpty(string) → boolean stabilisce se la stringa è vuota

equal(string, string) → boolean stabilisce se due stringhe sono uguali

Esercizio (cont.)

<i>osservazioni</i>	<i>Costruttore di s'</i>	
	<i>new()</i>	<i>add(s, c)</i>
<i>length(s')</i>	0	length(s)+1
<i>isEmpty(s')</i>	true	false
<i>append(t,s')</i>	t	add(append(t,s),c)

Esercizio (cont.)

equal(string, string) ☐ boolean

<i>Costruttore di t'</i>	<i>Costruttore di s'</i>	
	<i>new()</i>	<i>add(s, c)</i>
<i>new()</i>	true	false
<i>add(t, h)</i>	false	if h=c then equal(t, s) else false

Esercizio

Progettare il dato astratto *Conto Con Fido* che consente di rappresentare dei conti correnti bancari per i quali è permesso uno scoperto. Si deve poter limitare lo scoperto tramite concessione del fido. Le operazioni ammesse per questo dato astratto sono:

conto (account): apre un nuovo conto corrente bancario definendo saldo iniziale e massimo scoperto ammesso

saldo (balance): riporta il saldo del conto

deposita (deposit): deposita una somma sul conto

preleva (withdraw): preleva denaro dal conto

concediFido (setOverdraftLimit): definisce il limite massimo dello scoperto

fidoConcesso (getOverdraftLimit): restituisce il limite massimo dello scoperto

Esercizio (cont.)

Specifica sintattica

sorts: contocorrente, saldo, denaro, scoperto;

operations:

conto(saldo, scoperto) -> contocorrente

saldo(contocorrente) -> saldo

deposita(contocorrente, denaro) -> contocorrente

preleva(contocorrente, denaro) -> contocorrente

concediFido(contocorrente, scoperto) -> contocorrente

fidoConcesso(contocorrente) -> scoperto

Esercizio (cont.)

Specifica semantica

Il dominio *contocorrente* è l'insieme delle coppie $(s, l) \in \mathbf{R} \times \mathbf{R}^+$, tali che $s \geq -l$. Il dominio *saldo* corrisponde all'insieme dei numeri reali $\mathbf{R}$, mentre i domini *scoperto* e *denaro* corrispondono ad $\mathbf{R}^+$.

Pertanto potremo disporre sui vari domini di tipiche operazioni algebriche, e in particolare:

- due operazioni binarie: “+” e “-”
- operatore unario: “-”

nonché di una relazione binaria “<” e di una costante “0”.

N.B.: formalmente avremmo dovuto indicare che “+” è definito sui domini *saldo* e *scoperto* e restituisce un *saldo*, etc.,

Esercizio (cont.)

N.B.: Gli operatori aggiuntivi che sono stati specificati non fanno riferimento al dominio *contocorrente* relativo al dato astratto che si vuole definire. Per questo non è necessario specificarli.

Diversamente, se fosse stata ipotizzata la disponibilità di un operatore del dominio *contocorrente*, avremmo dovuto successivamente indicarne la semantica al pari degli altri operatori indicati nella specifica sintattica.

Esercizio (cont.)

Possiamo identificare un solo costruttore (*conto*). Gli altri operatori sono tutte osservazioni.

<i>osservazioni</i>	<i>Costruttore di c</i>
	<i>conto(s,l)</i>
<i>saldo(c)</i>	s
<i>deposita(c,d')</i>	conto(s+d',l)
<i>preleva(c,d')</i>	if s-d'<-l then error else conto(s-d',l)
<i>concediFido(c,l')</i>	if s<-l' then error else conto(s,l')
<i>fidoConcesso(c)</i>	1

Esercizio (cont.)

Specifica semantica

declare **c**: contocorrente, **s**: saldo, **d**: denaro, **l**, **l'**: scoperto;

saldo(conto(s,l))=s

deposita(conto(s,l), d) = conto(s+d,l)

preleva(conto(s,l), d) = **if** s-d<-l **then** error **else** conto(s-d,l)

concediFido(conto(s,l),l') = **if** s<-l' **then** error **else** conto(s,l')

fidoConcesso(conto(s,l))=l

Esercizio (cont.)

Utilizzando le precedenti specifiche è possibile dimostrare la seguente equazione:

$$\text{preleva}(\text{deposita}(c, d), d) = c$$

Dim.: Sia $c = \text{conto}(s, l)$

$$\text{preleva}(\text{deposita}(\text{conto}(s, l), d), d) = \text{preleva}(\text{conto}(s+d, l), d)$$

per definizione di *deposita* ($\text{deposita}(\text{conto}(s, l), d) = \text{conto}(s+d, l)$)

Inoltre:

$$\text{preleva}(\text{conto}(s+d, l), d) = \text{conto}((s+d)-d, l)$$

per definizione di *preleva* ($\text{preleva}(\text{conto}(s, l), d) = \text{if } s-d < -l \text{ then error else } \text{conto}(s-d, l)$).

$$\text{Ma } \text{conto}((s+d)-d, l) = \text{conto}(s, l) = c$$